Error Handling

Module 1.4 — Session 7 — 2026-03-22

`Result<T, E>` — Errors as Values

Rust has no exceptions. Fallible functions return `Result`:

```
enum Result<T, E> {
    Ok(T),   // success with value
    Err(E),  // failure with error
}
```

The error is in the type signature — the compiler won't let you ignore it. Analogous to Haskell's `Either a b` (`Left` = error, `Right` = success), but with explicit naming.

The ? Operator

Sugar for "unwrap or propagate":

```
// These are equivalent:
let size = validate_dimensions(w, h)?;

let size = match validate_dimensions(w, h) {
    Ok(s) => s,
    Err(e) => return Err(e),
};
```

`?` can only be used in functions that return `Result` (or `Option`). It's an early return — no hidden control flow, no stack unwinding.

`main` can return `Result<(), E>` if you want `?` at the top level. On `Err`, Rust prints the error via `Debug` and exits with code 1.

Custom Error Types

`String` errors can't be matched programmatically. Use an enum:

```
#[derive(Debug)]
enum WorldError {
    ZeroDimensions,
    TooManyTiles { total: usize },
}
```

The caller can now match on specific failure modes, and the compiler checks exhaustiveness.

`Display` and `std::error::Error`

The idiomatic error type implements three things:

Trait	How
`Debug`	Derive it — `#[derive(Debug)]`
`Display`	Hand-write — human-readable messages
`Error`	`impl Error for MyError {}` — requires `Display + Debug`

```
impl fmt::Display for WorldError {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            WorldError::ZeroDimensions =>
                write!(f, "World dimensions cannot be zero."),
            WorldError::TooManyTiles { total } =>
                write!(f, "Too many tiles: {}. Max is 10000.", total),
        }
    }
}
```

Note: `main` returning `Err` prints via `Debug`, not `Display`. To get `Display` output, handle the error yourself with `eprintln!("{e}")`.

`panic!` vs `Result`

`panic!`	`Result`
Bug — invariant violated	Expected runtime failure
Unrecoverable	Caller decides how to handle
Index out of bounds, broken logic	Invalid input, file not found, network error

`unwrap` and `expect`

Both extract the inner value from `Ok`/`Some`, panicking on `Err`/`None`:

```
let val = some_result.unwrap();        // panics with generic message
let val = some_result.expect("why");   // panics with your message
```

`.expect()` documents the assumption — why you believe this can't fail. Prefer it over `.unwrap()` in non-throwaway code.

> **Key Insight**
>
> The hierarchy for handling a `Result`:
> 1. **`?`** — propagate to caller
> 2. **`match`** — handle here
> 3. **`.expect("reason")`** — assert infallibility, document why
> 4. **`.unwrap()`** — assert infallibility (fine in tests/prototypes)

Library vs Application Code

> **Design principle**
>
> Library functions return `Result` — they report errors but never decide policy. Application code (`main`) decides: print and exit? Retry? Fall back? This separation keeps libraries composable.